Midnight Shielded SDK — Annotated Code Reference

Purpose: A comprehensive, study-oriented reference of the Midnight SDK source code for shielded (Zswap) operations. Read this alongside `SHIELDED_BALANCE_DEEP_DIVE.md` — that document explains the *concepts*; this document shows you the *actual code*.

Source codebases:

- **Rust** — `midnight-ledger/` (the cryptographic engine)
- **TypeScript** — `midnight-wallet/` (the wallet SDK)
- **Kotlin** — `kuira-android-wallet/` (our Android implementation)

Table of Contents

Chapter 1: The Two Curves

Midnight uses a **curve pair**: a main (outer) curve and an embedded (inner) curve. This is fundamental — every piece of cryptography builds on these two algebraic structures.

Why Two Curves?

The outer curve provides the main prime field used by the zero-knowledge proof system. The embedded curve lives *inside* that field — its coordinates are elements of the outer field. This lets the proof system reason about curve operations efficiently.

The Outer Curve: BLS12-381

```
// Source: transient-crypto/src/curve.rs

/// The outer, main curve
pub mod outer {
    /// The base prime field, used to represent curve points
    pub type Base = midnight_curves::Fp;
    /// The scalar prime field, used in circuit
    pub type Scalar = midnight_curves::Fq;
    /// The affine representation of a curve point
    pub type Affine = midnight_curves::G1Affine;
}
```

The outer curve is BLS12-381. Its scalar field `Fq` is what Midnight calls `Fr` — the primary field element type used across the entire system.

The Embedded Curve: Jubjub

```
// Source: transient-crypto/src/curve.rs

/// The embedded / cycle curve, used in-circuit mainly
pub mod embedded {
    /// The base prime field, used to represent curve points;
    /// the scalar of outer
    pub type Base = midnight_curves::Fq;
    /// The scalar prime field, used in embedded proofs
    pub type Scalar = midnight_curves::Fr;
    /// The affine representation of a curve point of the
    /// relevant cryptographic subgroup.
    pub type Affine = midnight_curves::JubjubSubgroup;
}
```

The embedded curve is Jubjub — a Twisted Edwards curve whose base field *is* the outer scalar field. This is the key relationship: `embedded::Base == outer::Scalar`. Points on Jubjub can be represented as pairs of outer field elements.

The Wrapper Types

Midnight wraps the raw curve types in newtypes for safety and ergonomics:

```
// Source: transient-crypto/src/curve.rs

/// An element of our primary prime field.
pub struct Fr(pub outer::Scalar);

/// An element of our embedded prime field.
pub struct EmbeddedFr(pub embedded::Scalar);

/// An element in the embedded elliptic curve.
pub struct EmbeddedGroupAffine(pub embedded::Affine);
```

`Fr` is the workhorse type. It appears everywhere: hash outputs, field representations, encryption ciphertexts, Poseidon inputs. Think of it as "a number in the main field."

`EmbeddedFr` is the scalar for the Jubjub curve. It's used for Pedersen commitment randomness and encryption secret keys.

`EmbeddedGroupAffine` is a point on Jubjub. It's used for Pedersen commitments, encryption public keys, and the El Gamal challenge point.

Key Constants

```
// Source: transient-crypto/src/curve.rs

/// The number of bits required to represent Fr.
pub const FR_BITS: usize = 254;  // BLS12-381 scalar field

/// The number of bytes required to represent Fr.
pub const FR_BYTES: usize = 32;  // ceil(254/8)

/// The number of bytes which can fit in an Fr.
pub const FR_BYTES_STORED: usize = 31;  // FR_BYTES - 1
```

The critical insight: although `Fr` takes 32 bytes to serialize, only 31 bytes of *arbitrary* data can safely be stored in one field element (because the field modulus is less than 2^256). This means:

- **A 32-byte hash → 2 field elements** (31 bytes in one, 1 byte in the other)
- **A u128 → 1 field element** (16 bytes fits easily)
- **A nonce (32 bytes) → 2 field elements**

Converting Between Fr and EmbeddedFr

```
// Source: transient-crypto/src/curve.rs

impl TryFrom<EmbeddedFr> for Fr {
    type Error = ();
    fn try_from(val: EmbeddedFr) -> Result<Fr, Self::Error> {
        Fr::from_le_bytes(&val.as_le_bytes()).ok_or(())
```

```
    }
}

impl TryFrom<Fr> for EmbeddedFr {
    type Error = ();
    fn try_from(val: Fr) -> Result<EmbeddedFr, Self::Error> {
        EmbeddedFr::from_le_bytes(&val.as_le_bytes()).ok_or(())
    }
}
```

Both conversions are fallible because the two fields have different moduli. The embedded modulus is smaller, so every `EmbeddedFr` fits in `Fr`, but not vice versa.

EmbeddedGroupAffine: Key Operations

```
// Source: transient-crypto/src/curve.rs

impl EmbeddedGroupAffine {
    /// Returns the primary generator of the embedded curve.
    pub fn generator() -> Self {
        EmbeddedGroupAffine(embedded::Affine::generator())
    }

    /// Returns the identity element for curve addition.
    pub fn identity() -> Self {
        EmbeddedGroupAffine(embedded::Affine::identity())
    }

    /// Retrieves the curve point's affine x coordinate.
    pub fn x(&self) -> Option<Fr> { /* ... */ }

    /// Retrieves the curve point's affine y coordinate.
    pub fn y(&self) -> Option<Fr> { /* ... */ }
}
```

Note that `x()` and `y()` return `Fr` (the outer field), confirming that Jubjub points live in the outer field.

Chapter 2: Field Element Encoding

How do you turn a piece of data (a hash, a number, a coin) into field elements for use in Poseidon hashing or zero-knowledge proofs? The answer: the `FieldRepr` trait.

The Core Encoding Rule: 31 Bytes Per Element

```
// Source: transient-crypto/src/repr.rs

impl FieldRepr for [u8] {
    fn field_repr<W: MemWrite<Fr>>(&self, writer: &mut W) {
        let mut slice = self;
```

```
        while !slice.is_empty() {
            let len = slice.len();
            let stray = len % FR_BYTES_STORED;      // remainder bytes
            if stray != 0 {
                // Encode the "stray" bytes (remainder) first
                writer.write(&[Fr::from_le_bytes(&slice[len - stray..])
                    .expect("Must fall in storable byte range")]);
                slice = &slice[..len - stray];
            } else {
                // Full 31-byte chunks
                let start = len - usize::min(FR_BYTES_STORED, len);
                writer.write(&[Fr::from_le_bytes(&slice[start..])
                    .expect("Must fall in storable byte range")]);
                slice = &slice[..start];
            }
        }
    }

    fn field_size(&self) -> usize {
        self.len().div_ceil(FR_BYTES_STORED)  // ceil(n/31)
    }
}
```

How a 32-byte value becomes 2 field elements:

For `[u8; 32]` (like a `HashOutput`):

1. First, encode the "stray" byte: `32 % 31 = 1`, so the last byte becomes the first `Fr`
2. Then, encode the remaining 31 bytes as the second `Fr`

So `FromFieldRepr for [u8; 32]`:

```
// Source: transient-crypto/src/repr.rs

impl FromFieldRepr for [u8; 32] {
    const FIELD_SIZE: usize = 2;
    fn from_field_repr(repr: &[Fr]) -> Option<Self> {
        if repr.len() != 2 { return None; }
        let repr0 = repr[0].0.to_bytes_le();
        let repr1 = repr[1].0.to_bytes_le();
        // repr[0] holds 1 stray byte (in lowest position)
        // repr[1] holds 31 bytes
        let mut res = [0u8; 32];
        res[31..].copy_from_slice(&repr0[..1]);
        res[0..31].copy_from_slice(&repr1[..31]);
        Some(res)
    }
}
```

Numeric Types: Direct Embedding

```
// Source: transient-crypto/src/repr.rs

// u128, u64, u32, u16, u8, i128, i64, i32, i16, i8, Fr, bool
// All map to exactly 1 field element:
```

```rust
impl FieldRepr for $ty {
    fn field_repr<W: MemWrite<Fr>>(&self, writer: &mut W) {
        writer.write(&[Fr::from(*self)]);
    }
    fn field_size(&self) -> usize { 1 }
}
```

A `u128` value fits in a single `Fr` because 128 bits < 254 bits.

HashOutput → 2 Field Elements

```rust
// Source: transient-crypto/src/hash.rs

impl FieldRepr for HashOutput {
    fn field_repr<W: MemWrite<Fr>>(&self, writer: &mut W) {
        self.0.field_repr(writer);  // delegates to [u8; 32]
    }
    fn field_size(&self) -> usize {
        self.0.field_size()  // = 2
    }
}
```

Summary Table: Field Element Sizes

Type	Field Elements	Why
`u128`	1	128 bits < 254-bit field
`bool`	1	0 or 1
`Fr`	1	Already a field element
`HashOutput` ([u8; 32])	2	32 bytes > 31 storable bytes
`Nonce`	2	Wraps `HashOutput`
`ShieldedTokenType`	2	Wraps `HashOutput`
`PublicKey` (coin)	2	Wraps `HashOutput`
`SecretKey` (coin)	2	Wraps `HashOutput`
`CoinInfo` (nonce + type + value)	2 + 2 + 1 = **5**	Sum of fields
`EmbeddedGroupAffine`	2	x and y coordinates, each 1 `Fr`
`Pedersen`	2	Wraps `EmbeddedGroupAffine`

Chapter 3: Two Hash Functions

Midnight uses two fundamentally different hash functions, each with a specific role.

Persistent Hash: SHA-256

```rust
// Source: base-crypto/src/hash.rs

/// A hash function that is guaranteed for long-term support.
pub fn persistent_hash(a: &[u8]) -> HashOutput {
    HashOutput(Sha256::digest(a).into())
}
```

Used for: Commitments, nullifiers, coin public key derivation, coin secret key derivation — anything that must remain stable across hard-forks.

The streaming variant:

```rust
// Source: base-crypto/src/hash.rs

pub struct PersistentHashWriter(Sha256);

impl MemWrite<u8> for PersistentHashWriter {
    fn write(&mut self, buf: &[u8]) {
        self.0.update(buf);
    }
}

impl PersistentHashWriter {
    pub fn new() -> Self { Default::default() }
    pub fn finalize(self) -> HashOutput {
        HashOutput(self.0.finalize().into())
    }
}
```

The `persistent_commit` helper:

```rust
// Source: base-crypto/src/hash.rs

pub fn persistent_commit<T: BinaryHashRepr + ?Sized>(
    value: &T,
    opening: HashOutput,
) -> HashOutput {
    let mut writer = PersistentHashWriter::new();
    opening.binary_repr(&mut writer);      // opening first
    value.binary_repr(&mut writer);        // then the value
    writer.finalize()
}
```

Transient Hash: Poseidon

```rust
// Source: transient-crypto/src/hash.rs

/// An efficient hash function that may be changed on hard-forks.
pub fn transient_hash(elems: &[Fr]) -> Fr {
    let h = <PoseidonChip<outer::Scalar> as HashCPU<
        outer::Scalar, outer::Scalar
    >>::hash(
        &elems.iter().map(|x| x.0).collect::<Vec<_>>(),
    );
    Fr(h)
}
```

Used for: Encryption key derivation, CTR keystream, Pedersen generator derivation (`hash_to_curve`), anything that needs to be SNARK-friendly.

Key difference: Poseidon operates on field elements (`Fr`), not bytes. SHA-256 operates on bytes.

The `transient_commit` helper:

```rust
// Source: transient-crypto/src/hash.rs

pub fn transient_commit<T: FieldRepr + ?Sized>(
    value: &T,
    opening: Fr,
) -> Fr {
    let mut preimage = vec![opening];
    value.field_repr(&mut preimage);
    transient_hash(&preimage)
}
```

hash_to_curve: Mapping Field Elements to Curve Points

```rust
// Source: transient-crypto/src/hash.rs

pub fn hash_to_curve<T: FieldRepr + ?Sized>(
    value: &T,
) -> EmbeddedGroupAffine {
    let preimage = value.field_vec()
        .into_iter()
        .map(|f| f.0)
        .collect::<Vec<_>>();
    // Uses Poseidon-based hash-to-curve gadget
    let point = HashToCurveGadget::hash_to_curve(&preimage);
    EmbeddedGroupAffine(point)
}
```

This is critical for Pedersen commitments: it converts a token type into a curve generator `H(type)` .

Converting Between Hash Domains

```rust
// Source: transient-crypto/src/hash.rs

/// Transforms persistent hash → transient (lossy, keeps 31 bytes)
pub fn degrade_to_transient(persistent: HashOutput) -> Fr {
    persistent.field_vec()[1]  // takes the 31-byte chunk
}

/// Transforms transient hash → persistent (pads with zero)
pub fn upgrade_from_transient(transient: Fr) -> HashOutput {
    let mut res = [0u8; 32];
    res[..FR_BYTES_STORED].copy_from_slice(
        &transient.as_le_bytes()[..FR_BYTES_STORED]
    );
    HashOutput(res)
}
```

Chapter 4: Serialization Traits

Midnight has two parallel serialization systems, each feeding a different hash function.

BinaryHashRepr → feeds SHA-256

```rust
// Source: base-crypto/src/repr.rs

pub trait BinaryHashRepr {
    /// Writes out the binary representation into a writer.
    fn binary_repr<W: MemWrite<u8>>(&self, writer: &mut W);
    /// The size of the binary representation.
    fn binary_len(&self) -> usize;
    /// Convenience: writes to a Vec<u8>
    fn binary_vec(&self) -> Vec<u8> { /* ... */ }
}
```

Key implementations:

```rust
// Integers: little-endian bytes
impl BinaryHashRepr for u128 {
    fn binary_repr<W: MemWrite<u8>>(&self, writer: &mut W) {
        writer.write(&self.to_le_bytes());  // 16 bytes
    }
    fn binary_len(&self) -> usize { 16 }
}

// Bool: single byte
impl BinaryHashRepr for bool {
    fn binary_repr<W: MemWrite<u8>>(&self, writer: &mut W) {
        writer.write(&[*self as u8]);  // 0x00 or 0x01
```

```
    }
    fn binary_len(&self) -> usize { 1 }
}

// Tuples: fields concatenated in order
impl<A: BinaryHashRepr, B: BinaryHashRepr> BinaryHashRepr for (A, B) {
    fn binary_repr<W: MemWrite<u8>>(&self, writer: &mut W) {
        self.0.binary_repr(writer);
        self.1.binary_repr(writer);
    }
}

// Fr: 32 bytes little-endian
impl BinaryHashRepr for Fr {
    fn binary_repr<W: MemWrite<u8>>(&self, writer: &mut W) {
        writer.write(&self.as_le_bytes())  // 32 bytes
    }
    fn binary_len(&self) -> usize { FR_BYTES }  // 32
}
```

FieldRepr → feeds Poseidon

```
// Source: transient-crypto/src/repr.rs

pub trait FieldRepr {
    /// Writes out self as a sequence of Fr elements.
    fn field_repr<W: MemWrite<Fr>>(&self, writer: &mut W);
    /// The number of field elements this produces.
    fn field_size(&self) -> usize;
    /// Convenience: writes to a Vec<Fr>
    fn field_vec(&self) -> Vec<Fr> { /* ... */ }
}
```

The `#[derive(BinaryHashRepr)]` Macro

The Midnight codebase uses derive macros extensively. When you see:

```
#[derive(BinaryHashRepr)]
pub struct Info {
    pub nonce: Nonce,
    pub type_: ShieldedTokenType,
    pub value: u128,
}
```

This generates a `binary_repr` that serializes `nonce || type_ || value` — fields in **declaration order**. This is critical for commitment and nullifier computation.

Chapter 5: Key Derivation

Starting from a 32-byte seed, Midnight derives two independent key pairs.

The Seed

```rust
// Source: zswap/src/keys.rs

#[derive(Zeroize, ZeroizeOnDrop)]
pub struct Seed([u8; 32]);

impl Seed {
    pub fn random<T: Rng + CryptoRng>(rng: &mut T) -> Seed {
        let mut out: [u8; 32] = [0; 32];
        rng.fill_bytes(&mut out);
        Seed(out)
    }
}
```

Note the `Zeroize` and `ZeroizeOnDrop` — the seed is wiped from memory when no longer needed.

Coin Secret Key: Single SHA-256

```rust
// Source: zswap/src/keys.rs

pub fn derive_coin_secret_key(self: &Seed) -> coin::SecretKey {
    let domain_separator = b"midnight:csk";
    let mut hash_writer = PersistentHashWriter::new();
    MemWrite::write(&mut hash_writer, domain_separator);
    MemWrite::write(&mut hash_writer, &self.0);
    let hash = hash_writer.finalize();
    coin::SecretKey(hash)
}
```

Formula: `coin_sk = SHA-256("midnight:csk" || seed)`

The coin secret key is a `HashOutput` (32 bytes). Simple, one-shot derivation.

Coin Public Key: Another SHA-256

```rust
// Source: coin-structure/src/coin.rs

impl SecretKey {
    pub fn public_key(&self) -> PublicKey {
        let mut data = Vec::with_capacity(21 + 32);
        data.extend(b"midnight:zswap-pk[v1]");
        self.binary_repr(&mut data);
        PublicKey(persistent_hash(&data))
```

```
    }
}
```

Formula: `coin_pk = SHA-256("midnight:zswap-pk[v1]" || coin_sk)`

Both coin keys are hash outputs, not curve points. This is unusual — there's no elliptic curve discrete log relationship between them. The binding is purely through hashing.

Encryption Secret Key: Multi-Round KDF

```rust
// Source: zswap/src/keys.rs

pub fn derive_encryption_secret_key(self: &Seed) -> encryption::SecretKey {
    const DOMAIN_SEPARATOR: &[u8; 12] = b"midnight:esk";
    const NUMBER_OF_BYTES: usize = 64;
    let mut raw_bytes = self.sample_bytes(NUMBER_OF_BYTES, DOMAIN_SEPARATOR);
    let mut raw_bytes_arr: [u8; 64] = raw_bytes.clone().try_into().unwrap();

    raw_bytes.zeroize();
    let res = encryption::SecretKey::from_uniform_bytes(&raw_bytes_arr);
    raw_bytes_arr.zeroize();
    res
}
```

This derives 64 bytes, then reduces them modulo the Jubjub scalar field using `from_uniform_bytes` (which uses wide reduction to avoid bias).

The sample_bytes KDF

```rust
// Source: zswap/src/keys.rs

pub fn sample_bytes(
    &self,
    no_of_bytes: usize,
    domain_separator: &[u8],
) -> Vec<u8> {
    let hash_bytes = PERSISTENT_HASH_BYTES;  // 32
    let rounds = no_of_bytes.div_ceil(hash_bytes);
    let mut res: Vec<u8> = Vec::new();
    for round in 0..rounds {
        let mut outer_writer = PersistentHashWriter::new();
        MemWrite::write(&mut outer_writer, domain_separator);
        MemWrite::write(&mut outer_writer, &{
            let mut inner_writer = PersistentHashWriter::new();
            MemWrite::write(
                &mut inner_writer,
                &((round as u64).to_le_bytes()),
            );
            MemWrite::write(&mut inner_writer, &self.0);
            inner_writer.finalize().0
        });
        let round_hash = outer_writer.finalize();
        let bytes_to_add = hash_bytes.min(no_of_bytes - round * 32);
        res.extend_from_slice(&round_hash.0[0..bytes_to_add])
```

```
    }
    res
}
```

For 64 bytes, this runs 2 rounds:

Round 0:

```
inner = SHA-256(0u64_le || seed)
output = SHA-256("midnight:esk" || inner)
→ 32 bytes
```

Round 1:

```
inner = SHA-256(1u64_le || seed)
output = SHA-256("midnight:esk" || inner)
→ 32 bytes
```

Concatenate → 64 bytes → `EmbeddedFr::from_uniform_bytes`

Encryption Public Key: Curve Scalar Multiplication

```rust
// Source: transient-crypto/src/encryption.rs

impl SecretKey {
    pub fn public_key(&self) -> PublicKey {
        PublicKey(EmbeddedGroupAffine::generator() * self.0)
    }
}
```

Formula: `enc_pk = G * enc_sk` (Jubjub generator times the secret scalar)

Unlike the coin key pair (hash-based), the encryption key pair uses standard elliptic curve discrete log.

The SecretKeys Aggregate

```rust
// Source: zswap/src/keys.rs

#[derive(Clone, Zeroize, ZeroizeOnDrop)]
pub struct SecretKeys {
    pub coin_secret_key: coin::SecretKey,
    pub encryption_secret_key: encryption::SecretKey,
}

impl From<Seed> for SecretKeys {
    fn from(seed: Seed) -> Self {
        SecretKeys {
            coin_secret_key: seed.derive_coin_secret_key(),
            encryption_secret_key: seed.derive_encryption_secret_key(),
        }
    }
```

```
}

impl SecretKeys {
    pub fn coin_public_key(&self) -> coin::PublicKey {
        self.coin_secret_key.public_key()
    }

    pub fn enc_public_key(&self) -> encryption::PublicKey {
        self.encryption_secret_key.public_key()
    }

    pub fn try_decrypt(&self, msg: &CoinCiphertext) -> Option<CoinInfo> {
        self.encryption_secret_key.decrypt(&msg.clone().into())
    }
}
```

Key Derivation Summary

```
seed (32 bytes)
├── coin_sk = SHA-256("midnight:csk" || seed)           → HashOutput (32 bytes)
│   └── coin_pk = SHA-256("midnight:zswap-pk[v1]" || coin_sk)  → HashOutput (32
bytes)
└── enc_sk = sample_bytes(64, "midnight:esk") → from_uniform_bytes  → EmbeddedFr
(Jubjub scalar)
    └── enc_pk = G * enc_sk                             → EmbeddedGroupAffine
(Jubjub point)
```

Chapter 6: The Shielded Coin

A coin is the fundamental unit of value in the shielded (Zswap) system.

CoinInfo: The Unqualified Coin

```
// Source: coin-structure/src/coin.rs

#[derive(FieldRepr, FromFieldRepr, BinaryHashRepr, Serializable)]
pub struct Info {
    pub nonce: Nonce,               // 32-byte random value
    pub type_: ShieldedTokenType,   // 32-byte token identifier
    pub value: u128,                // amount
}
```

Three fields, all you need:

- **nonce** — Random, makes each coin unique even with same type/value
- **type_** — Which token this is (NIGHT has hash of all zeros)
- **value** — How many units

Field element count: `Nonce(2) + ShieldedTokenType(2) + u128(1) = 5`
Binary size: `Nonce(32) + ShieldedTokenType(32) + u128(16) = 80 bytes`

QualifiedCoinInfo: Coin + Position

```rust
// Source: coin-structure/src/coin.rs

#[derive(FieldRepr, FromFieldRepr, BinaryHashRepr, Serializable)]
pub struct QualifiedInfo {
    pub nonce: Nonce,
    pub type_: ShieldedTokenType,
    pub value: u128,
    pub mt_index: u64,    // position in the Merkle tree
}
```

Once a coin is inserted into the Merkle tree, it gets an index. This qualified version is what the wallet actually stores.

Token Types

```rust
// Source: coin-structure/src/coin.rs

pub enum TokenType {
    Unshielded(UnshieldedTokenType),
    Shielded(ShieldedTokenType),
    Dust,
}

pub const NIGHT: UnshieldedTokenType =
    UnshieldedTokenType(HashOutput([0u8; 32]));
```

NIGHT (the native token) has a type hash of all zeros for the unshielded variant.

The Recipient and SenderEvidence Enums

These are critical for commitment and nullifier computation:

```rust
// Source: coin-structure/src/transfer.rs

pub enum Recipient {
    User(PublicKey),             // A user's coin public key
    Contract(ContractAddress),   // A smart contract address
}

pub enum SenderEvidence<'a> {
    User(Cow<'a, SecretKey>),      // The sender's coin secret key
    Contract(ContractAddress),      // A contract proving ownership
}
```

The `Cow<'a, SecretKey>` avoids unnecessary cloning of the secret key.

Binary Encoding in Commitments/Nullifiers

The commitment and nullifier functions encode the Recipient/SenderEvidence as a simple tuple:

```
// Source: coin-structure/src/coin.rs (inside commitment/nullifier)

// Recipient::User(pk)           → (true, pk.0).binary_repr()  → 0x01 ||
pk_32_bytes
// Recipient::Contract(addr)     → (false, addr.0).binary_repr() → 0x00 ||
addr_32_bytes

// SenderEvidence::User(sk)      → (true, sk.0).binary_repr()  → 0x01 ||
sk_32_bytes
// SenderEvidence::Contract(addr) → (false, addr.0).binary_repr() → 0x00 ||
addr_32_bytes
```

Both variants are the same binary length: **33 bytes** (1 byte tag + 32 byte key). The `true` / `false` tag byte distinguishes User from Contract.

Note: The `FieldRepr` (used for Poseidon hashing) has a *different* encoding that includes `BLANK_HASH` padding for fixed-width alignment. Don't confuse the two.

Chapter 7: Commitments and Nullifiers

These are the twin pillars of the shielded system. A commitment proves a coin exists. A nullifier proves a coin was spent.

Commitment Computation

```rust
// Source: coin-structure/src/coin.rs

impl Info {
    pub fn commitment(&self, recipient: &Recipient) -> Commitment {
        let mut data = Vec::with_capacity(21 + 32 + 32 + 16 + 1 + 32);
        data.extend(b"midnight:zswap-cc[v1]");
        self.binary_repr(&mut data);
        match &recipient {
            Recipient::User(d) =>
                (true, d.0).binary_repr(&mut data),
            Recipient::Contract(d) =>
                (false, d.0).binary_repr(&mut data),
        }
        Commitment(persistent_hash(&data))
    }
}
```

Formula:

```
commitment = SHA-256(
    "midnight:zswap-cc[v1]"    // 21 bytes domain separator
```

```
    || nonce                    // 32 bytes (BinaryHashRepr of Nonce)
    || type_                    // 32 bytes (BinaryHashRepr of ShieldedTokenType)
    || value                    // 16 bytes (u128 LE)
    || recipient_tag            //  1 byte  (true=User, false=Contract)
    || recipient_key            // 32 bytes (PublicKey or ContractAddress)
)                               // Total: 134 bytes
```

Nullifier Computation

```rust
// Source: coin-structure/src/coin.rs

impl Info {
    pub fn nullifier(&self, se: &SenderEvidence<'_>) -> Nullifier {
        let mut data = Vec::with_capacity(21 + 32 + 32 + 16 + 1 + 32);
        data.extend(b"midnight:zswap-cn[v1]");
        self.binary_repr(&mut data);
        match &se {
            SenderEvidence::User(d) =>
                (true, d.0).binary_repr(&mut data),
            SenderEvidence::Contract(d) =>
                (false, d.0).binary_repr(&mut data),
        }
        let res = Nullifier(persistent_hash(&data));
        data.zeroize();  // Wipe the secret key from memory!
        res
    }
}
```

Formula:

```
nullifier = SHA-256(
    "midnight:zswap-cn[v1]"    // 21 bytes domain separator
    || nonce                    // 32 bytes
    || type_                    // 32 bytes (BinaryHashRepr of ShieldedTokenType)
    || value                    // 16 bytes
    || sender_tag               //  1 byte  (true=User, false=Contract)
    || sender_evidence          // 32 bytes (SecretKey or ContractAddress)
)                               // Total: 134 bytes
```

Notice: `data.zeroize()` — the buffer containing the secret key is explicitly wiped.

The Critical Relationship

For a user-owned coin:

- **Commitment** uses `coin_pk` (public) → Anyone with the public key can verify
- **Nullifier** uses `coin_sk` (secret) → Only the owner can reveal the nullifier

When you spend a coin, you reveal its nullifier. The network checks that:

1. The nullifier hasn't been seen before (prevents double-spend)
2. The corresponding commitment exists in the Merkle tree (proves the coin is real)

But the network **cannot** link a nullifier back to its commitment without knowing the secret key.

Chapter 8: Pedersen Commitments

Pedersen commitments are used to hide transaction values while allowing the network to verify that inputs and outputs balance.

The Core Commit Function

```rust
// Source: transient-crypto/src/commitment.rs

pub struct Pedersen(pub EmbeddedGroupAffine);

impl Pedersen {
    /// Produces: g^r * H(type_)^v
    pub fn commit<T: FieldRepr + ?Sized>(
        type_: &T,
        v: &EmbeddedFr,
        r: &EmbeddedFr,
    ) -> Self {
        let h = hash_to_curve(type_);                // H(type_)
        let g = EmbeddedGroupAffine::generator();    // generator g
        let com = g * *r + h * *v;                   // g^r + H(type_)^v
        Pedersen(com)
    }
}
```

Mathematical formula: `Commit(type, v, r) = g^r · H(type)^v`

Where:

- `g` — The fixed Jubjub generator
- `H(type)` — A type-specific generator derived via `hash_to_curve`
- `v` — The value (as `EmbeddedFr`)
- `r` — Random blinding factor (as `EmbeddedFr`)

Homomorphic Properties

```rust
// Source: transient-crypto/src/commitment.rs

impl Add<Pedersen> for Pedersen {
    type Output = Pedersen;
    fn add(self, other: Self) -> Self {
        Pedersen(self.0 + other.0)
    }
}

impl Sub<Pedersen> for Pedersen {
    type Output = Pedersen;
```

```
    fn sub(self, other: Self) -> Self {
        Pedersen(self.0 - other.0)
    }
}

impl Neg for Pedersen {
    type Output = Pedersen;
    fn neg(self) -> Self {
        Pedersen(-self.0)
    }
}
```

This is what makes Pedersen commitments special: **you can add and subtract them** without revealing the values. If inputs commit to values that sum to the same as outputs, the commitments will also sum correctly:

```
Commit(type, v1, r1) + Commit(type, v2, r2) = Commit(type, v1+v2, r1+r2)
```

The network verifies `sum(input_commitments) - sum(output_commitments) = known_delta`.

Chapter 9: Encryption

When someone sends you a shielded coin, they encrypt the coin details so only you can read them. Midnight uses a custom El Gamal + CTR mode scheme operating on field elements.

Encrypt

```
// Source: transient-crypto/src/encryption.rs

impl PublicKey {
    pub fn encrypt<R: Rng + CryptoRng + ?Sized, T: FieldRepr>(
        &self,
        rng: &mut R,
        msg: &T,
    ) -> Ciphertext {
        // Step 1: Generate ephemeral scalar
        let y: EmbeddedFr = rng.r#gen();

        // Step 2: El Gamal challenge point
        let c = EmbeddedGroupAffine::generator() * y;  // c = G * y

        // Step 3: Shared secret
        let k_star = self.0 * y;  // K* = pk * y = (G*x)*y = G^(xy)

        // Step 4: Derive symmetric key via Poseidon
        let coords = if k_star.is_infinity() {
            (0.into(), 0.into())
        } else {
            (k_star.x().unwrap(), k_star.y().unwrap())
        };
        let k = transient_hash(&[coords.0, coords.1]);  // K = H(K*.x, K*.y)
```

```
        // Step 5: CTR mode encryption with field addition
        let ciph = once(0.into())            // Zero element for integrity check
            .chain(msg.field_vec())           // The actual message as field
elements
            .enumerate()
            .map(|(ctr, msg)|
                transient_hash(&[k, (ctr as u64).into()]) + msg  // CTR keystream
+ plaintext
            )
            .collect();

        Ciphertext { c, ciph }
    }
}
```

Step by step:

1. **Ephemeral key** `y` — Random Jubjub scalar, used once
2. **Challenge** `c = G·y` — Sent with the ciphertext, lets the recipient reconstruct the shared secret
3. **Shared secret** `K* = pk·y` — Only the recipient (who knows `x` such that `pk = G·x`) can compute `K* = c·x = G·xy`
4. **Symmetric key** `K = Poseidon(K*.x, K*.y)` — Converts the curve point to a field element key
5. **CTR keystream** — For each position `i`: `keystream[i] = Poseidon(K, i)`
6. **Encrypt** — `ciph[i] = keystream[i] + plain[i]` (field addition, not XOR)
7. **Zero element** — `ciph[0]` encrypts the value `0`, used to verify correct decryption

Decrypt

```
// Source: transient-crypto/src/encryption.rs

impl SecretKey {
    pub fn decrypt<T: FromFieldRepr>(&self, ciph: &Ciphertext) -> Option<T> {
        // Identity check: reject degenerate ciphertexts
        if ciph.c.is_identity() { return None; }

        // Step 1: Reconstruct shared secret
        let k_star = ciph.c * self.0;  // K* = c * x = (G*y) * x = G^(xy)

        // Step 2: Derive same symmetric key
        let coords = if k_star.is_infinity() {
            (0.into(), 0.into())
        } else {
            (k_star.x().unwrap(), k_star.y().unwrap())
        };
        let k = transient_hash(&[coords.0, coords.1]);

        // Step 3: CTR mode decryption
        let plain = ciph.ciph.iter().enumerate()
            .map(|(ctr, ciph)|
                *ciph - transient_hash(&[k, (ctr as u64).into()])
            )
            .collect::<Vec<_>>();

        // Step 4: Zero element check
        if plain.is_empty() || plain[0] != 0.into() {
```

```
            return None;  // Wrong key — decryption failed
        }

        // Step 5: Reconstruct the original type from field elements
        T::from_field_repr(&plain[1..])
    }
}
```

The zero element check is the key insight: if decryption with the wrong key produces a random field element instead of zero at position 0, we know it failed.

CoinCiphertext: Fixed-Size Variant

```
// Source: zswap/src/structure.rs

pub(crate) const COIN_CIPHERTEXT_LEN: usize = 6;

pub struct CoinCiphertext {
    pub c: EmbeddedGroupAffine,
    pub ciph: [Fr; COIN_CIPHERTEXT_LEN],  // Fixed 6 elements
}
```

Why 6 elements?

- 1 zero element (integrity check)
- 2 for nonce (32 bytes → 2 field elements)
- 2 for type_ (32 bytes → 2 field elements)
- 1 for value (u128 → 1 field element)
- Total: `1 + 2 + 2 + 1 = 6`

```
// Source: zswap/src/structure.rs

impl CoinCiphertext {
    pub fn new<R: Rng + CryptoRng + ?Sized>(
        rng: &mut R,
        coin: &CoinInfo,
        pk: encryption::PublicKey,
    ) -> CoinCiphertext {
        pk.encrypt(rng, coin)       // Generic encrypt
            .try_into()             // Convert Vec<Fr> → [Fr; 6]
            .expect("ciphertext should have ciphertext length")
    }
}
```

Chapter 10: The Merkle Tree

The Merkle tree stores all coin commitments. When you spend a coin, you must prove your commitment exists in this tree (via a Merkle path) without revealing *which* commitment.

Structure

```
// Source: transient-crypto/src/merkle_tree.rs

/// Sparse, fixed-depth Merkle trees.

/// The domain separator used in leaf commitments.
pub const LEAF_HASH_DOMAIN_SEP: &[u8] = b"mdn:lh";
```

The tree is sparse and fixed-depth. Most nodes are "stubs" (placeholder hashes for empty subtrees). The key operations:

- `update_hash(index, hash, aux)` — Insert a commitment at a position
- `collapse(start, end)` — Mark a range as irrelevant (saves memory)
- `rehash()` — Recompute all intermediate hashes up to the root
- `path_for_leaf(index)` — Generate a Merkle proof for a leaf
- `root()` — Get the current root hash

Why Collapse?

When processing events, if a coin isn't yours (decryption fails), you don't need to store its full path. The wallet calls `collapse()` to replace the subtree with just its hash, saving memory while keeping the root valid.

Chapter 11: ZswapLocalState

This is the wallet's local state — the "brain" that tracks your coins, pending operations, and the Merkle tree.

The State Structure

```
// Source: zswap/src/local.rs

pub struct State<D: DB> {
    pub coins: Map<Nullifier, QualifiedCoinInfo, D>,
    pub pending_spends: Map<Nullifier, QualifiedCoinInfo, D>,
    pub pending_outputs: Map<Commitment, CoinInfo, D>,
    pub merkle_tree: MerkleTree<(), D>,
    pub first_free: u64,
}
```

Five fields:

Field	Key	Value	Purpose
`coins`	Nullifier	QualifiedCoinInfo	Your confirmed coins

Field	Key	Value	Purpose
`pending_spends`	Nullifier	QualifiedCoinInfo	Coins you've submitted to spend but haven't been confirmed
`pending_outputs`	Commitment	CoinInfo	Coins you expect to receive (you pre-registered them via `watch_for`)
`merkle_tree`	—	—	Local view of the global commitment tree
`first_free`	—	u64	Next available index in the tree

Balance Calculation

```
available_balance = sum(coins.values().value) -
sum(pending_spends.values().value)
```

Coins that are both in `coins` AND in `pending_spends` exist simultaneously — `spend()` copies, not moves.

The spend() Method

```rust
// Source: zswap/src/local.rs

pub fn spend<R: Rng + CryptoRng + ?Sized>(
    &self,
    rng: &mut R,
    secret_keys: &SecretKeys,
    coin: &QualifiedCoinInfo,
    segment: Option<u16>,
) -> Result<(State<D>, Input<ProofPreimage, D>), OfferCreationFailed> {
    self.spend_from_tree(
        rng, secret_keys, coin, segment, &self.merkle_tree.clone()
    )
}

fn spend_from_tree<R: Rng + CryptoRng + ?Sized>(
    &self,
    rng: &mut R,
    secret_keys: &SecretKeys,
    coin: &QualifiedCoinInfo,
    segment: Option<u16>,
    tree: &MerkleTree<(), D>,
) -> Result<(State<D>, Input<ProofPreimage, D>), OfferCreationFailed> {
    let inp = Input::new_from_secret_key(
        rng, coin, segment,
        SenderEvidence::User(Cow::Borrowed(&secret_keys.coin_secret_key)),
        tree,
    )?;
    let res = State {
        pending_spends: self.pending_spends.insert(inp.nullifier, *coin),
        ..self.clone()
    };
```

```
    Ok((res, inp))
}
```

Critical behavior: `spend()` adds to `pending_spends` but does **NOT** remove from `coins`. The coin remains in `coins` until the blockchain confirms the spend via an `ZswapInput` event.

The apply() Method (Local Offer Processing)

```
// Source: zswap/src/local.rs

pub fn apply<P: Storable<D>>(
    &self,
    secret_keys: &SecretKeys,
    tx: &Offer<P, D>,
) -> State<D> {
    let mut res = self.clone();

    // Process outputs: try to find our coins
    for (coin_com, ciph) in tx.outputs.iter_deref()
        .map(|o| (&o.coin_com, &o.ciphertext))
        .chain(tx.transient.iter_deref()
            .map(|io| (&io.coin_com, &io.ciphertext)))
    {
        // Insert commitment into Merkle tree
        res.merkle_tree = res.merkle_tree
            .update_hash(res.first_free, coin_com.0, ());

        // Path 1: Try decryption
        if let Some(ci) = ciph.as_ref()
            .and_then(|ciph| secret_keys.try_decrypt(ciph))
        {
            let qci = ci.qualify(res.first_free);
            // Verify commitment matches
            if &ci.commitment(
                &Recipient::User(secret_keys.coin_public_key())
            ) == coin_com {
                res.coins = res.coins.insert(
                    CoinInfo::nullifier(&(&qci).into(),
                        &SenderEvidence::User(Cow::Borrowed(
                            &secret_keys.coin_secret_key))),
                    qci,
                );
                res.pending_outputs =
                    res.pending_outputs.remove(coin_com);
            }
        }
        // Path 2: Check pending_outputs
        else if let Some(coin) = res.pending_outputs.get(coin_com) {
            let qci = coin.qualify(res.first_free);
            res.coins = res.coins.insert(
                CoinInfo::nullifier(&(&qci).into(),
                    &SenderEvidence::User(Cow::Borrowed(
                        &secret_keys.coin_secret_key))),
                qci,
            );
            res.pending_outputs =
                res.pending_outputs.remove(coin_com);
```

```
        }
        // Path 3: Not our coin — collapse the tree node
        else {
            res.merkle_tree =
                res.merkle_tree.collapse(res.first_free, res.first_free);
        }

        res.first_free += 1;
    }

    // Process inputs: remove spent coins
    for nul in tx.inputs.iter_deref().map(|i| &i.nullifier)
        .chain(tx.transient.iter_deref().map(|io| &io.nullifier))
    {
        if let Some(_coin) = res.coins.get(nul) {
            res.coins = res.coins.remove(nul);
        }
        if let Some(_coin) = res.pending_spends.get(nul) {
            res.pending_spends = res.pending_spends.remove(nul);
        }
    }

    res.merkle_tree = res.merkle_tree.rehash();
    res
}
```

The apply_failed() Method

```
// Source: zswap/src/local.rs

pub fn apply_failed<P: Storable<D>>(&self, tx: &Offer<P, D>) -> State<D> {
    let mut res = self.clone();
    // Remove from pending_spends (the spend didn't go through)
    for nullifier in tx.inputs.iter_deref().map(|o| &o.nullifier)
        .chain(tx.transient.iter_deref().map(|io| &io.nullifier))
    {
        res.pending_spends = res.pending_spends.remove(nullifier);
    }
    // Remove from pending_outputs (the receive didn't go through)
    for coin_com in tx.outputs.iter_deref().map(|o| &o.coin_com)
        .chain(tx.transient.iter_deref().map(|io| &io.coin_com))
    {
        res.pending_outputs = res.pending_outputs.remove(coin_com);
    }
    res
}
```

The watch_for() Method

```
// Source: zswap/src/local.rs

pub fn watch_for(
    &self,
    coin_public_key: &coin::PublicKey,
```

```
    coin: &CoinInfo,
) -> State<D> {
    State {
        pending_outputs: self.pending_outputs.insert(
            coin.commitment(&Recipient::User(*coin_public_key)),
            *coin,
        ),
        ..self.clone()
    }
}
```

Pre-registers a coin you expect to receive. When the output event arrives, the wallet finds it in `pending_outputs` instead of needing to decrypt.

Chapter 12: Event Replay

When the wallet syncs, it processes blockchain events to update its local state. This is the core sync logic.

Event Types

```
// Source: ledger/src/events.rs

pub struct Event<D: DB> {
    pub source: EventSource,
    pub content: EventDetails<D>,
}

pub struct EventSource {
    pub transaction_hash: TransactionHash,
    pub logical_segment: u16,
    pub physical_segment: u16,
}

pub enum EventDetails<D: DB> {
    ZswapInput {
        nullifier: CoinNullifier,
        contract: Option<Sp<ContractAddress, D>>,
    },
    ZswapOutput {
        commitment: CoinCommitment,
        preimage_evidence: ZswapPreimageEvidence,
        contract: Option<Sp<ContractAddress, D>>,
        mt_index: u64,
    },
    ContractDeploy { /* ... */ },
    ContractLog { /* ... */ },
    ParamChange(/* ... */),
    DustInitialUtxo { /* ... */ },
    DustGenerationDtimeUpdate { /* ... */ },
    DustSpendProcessed { /* ... */ },
}
```

For shielded balance tracking, only `ZswapInput` and `ZswapOutput` matter.

ZswapPreimageEvidence: Three Ways to Learn About a Coin

```
// Source: ledger/src/events.rs

pub enum ZswapPreimageEvidence {
    Ciphertext(Box<CoinCiphertext>),      // Encrypted coin info
    PublicPreimage {                        // Plaintext coin info
        coin: CoinInfo,                     //   (used for contract-owned coins)
        recipient: Recipient,
    },
    None,                                   // No way to learn the preimage
}
```

The `try_with_keys` method attempts to recover coin info:

```
// Source: ledger/src/events.rs

impl ZswapPreimageEvidence {
    pub fn try_with_keys(
        &self,
        secret_keys: &ZswapSecretKeys,
    ) -> Option<CoinInfo> {
        match self {
            // Try to decrypt the ciphertext
            ZswapPreimageEvidence::Ciphertext(ciph) =>
                secret_keys.try_decrypt(ciph),

            // If it's a public preimage addressed to us, return it
            ZswapPreimageEvidence::PublicPreimage {
                coin,
                recipient: Recipient::User(pk),
            } if *pk == secret_keys.coin_public_key() =>
                Some(*coin),

            // Otherwise, nothing
            _ => None,
        }
    }
}
```

The Full replay_events_with_changes Implementation

```
// Source: ledger/src/semantics.rs

fn replay_events_with_changes<'a>(
    &self,
    secret_keys: &SecretKeys,
    events: impl IntoIterator<Item = &'a Event<D>>,
) -> Result<WithZswapStateChanges<Self>, EventReplayError> {
    use coin_structure::transfer::SenderEvidence;
```

```rust
let mut res = events.into_iter().try_fold(
    WithZswapStateChanges::new(self.clone()),
    |mut acc, event| match &event.content {

        // ── ZswapInput: A coin was spent ──────────────────
        EventDetails::ZswapInput {
            nullifier,
            contract: None,   // Only process user-owned inputs
        } => {
            // Track if this was our coin being spent
            let maybe_change = if acc.result.coins
                .contains_key(nullifier)
            {
                acc.result.coins.get(nullifier)
                    .map(|qci| ZswapStateChanges {
                        received_coins: vec![],
                        spent_coins: vec![*qci],
                        source: event.source.transaction_hash,
                    })
            } else { None };

            // Remove from BOTH maps
            acc.result.coins =
                acc.result.coins.remove(nullifier);
            acc.result.pending_spends =
                acc.result.pending_spends.remove(nullifier);

            Ok(acc.maybe_add_change(maybe_change))
        }

        // ── ZswapOutput: A new coin was created ───────────
        EventDetails::ZswapOutput {
            commitment,
            preimage_evidence,
            mt_index,
            ..
        } => {
            // Verify sequential insertion
            if *mt_index != acc.result.first_free {
                return Err(EventReplayError::NonLinearInsertion {
                    expected_next: acc.result.first_free,
                    received: *mt_index,
                    tree_name: "zswap commitment",
                });
            }

            // Insert into Merkle tree
            acc.result.merkle_tree = acc.result.merkle_tree
                .update_hash(*mt_index, commitment.0, ());
            acc.result.first_free += 1;

            let maybe_change =
            // Path 1: Check pending_outputs (pre-registered)
            if let Some(ci) = acc.result.pending_outputs
                .get(commitment)
            {
                let nullifier = ci.nullifier(
                    &SenderEvidence::User(Cow::Borrowed(
                        &secret_keys.coin_secret_key)));
                let qci = ci.qualify(*mt_index);
                acc.result.pending_outputs =
```

```
                        acc.result.pending_outputs.remove(commitment);
                    acc.result.coins =
                        acc.result.coins.insert(nullifier, qci);
                    Some(ZswapStateChanges {
                        received_coins: vec![qci],
                        spent_coins: vec![],
                        source: event.source.transaction_hash,
                    })
                }
                // Path 2: Try decryption / public preimage match
                else if let Some(ci) = preimage_evidence
                    .try_with_keys(secret_keys)
                {
                    let nullifier = ci.nullifier(
                        &SenderEvidence::User(Cow::Borrowed(
                            &secret_keys.coin_secret_key)));
                    let qci = ci.qualify(*mt_index);
                    acc.result.coins =
                        acc.result.coins.insert(nullifier, qci);
                    Some(ZswapStateChanges {
                        received_coins: vec![qci],
                        spent_coins: vec![],
                        source: event.source.transaction_hash,
                    })
                }
                // Path 3: Not our coin — collapse
                else {
                    acc.result.merkle_tree = acc.result
                        .merkle_tree
                        .collapse(*mt_index, *mt_index);
                    None
                };

                Ok(acc.maybe_add_change(maybe_change))
            }

            // ── All other events: skip ──────────────────────
            _ => Ok(acc),
        },
    )?;

    // Final rehash after all events
    res.result.merkle_tree = res.result.merkle_tree.rehash();

    Ok(res)
}
```

ZswapStateChanges: Tracking What Happened

```
// Source: ledger/src/zswap.rs

pub struct ZswapStateChanges {
    pub received_coins: Vec<QualifiedInfo>,
    pub spent_coins: Vec<QualifiedInfo>,
    pub source: TransactionHash,
}
```

```rust
pub struct WithZswapStateChanges<T> {
    pub changes: Vec<ZswapStateChanges>,
    pub result: T,
}
```

This wrapper lets the wallet know *which* coins were received or spent during event replay — useful for building transaction history.

Chapter 13: Transaction Building

When you want to spend a coin, you build an `Input`. When you want to create a coin, you build an `Output`.

Creating an Output (Sending a Coin)

```rust
// Source: zswap/src/construct.rs

impl<D: DB> Output<ProofPreimage, D> {
    pub(crate) fn new_for_recipient<R: Rng + CryptoRng + ?Sized>(
        rng: &mut R,
        coin: &CoinInfo,
        segment: Option<u16>,
        recipient: Recipient,
        ciphertext: Option<CoinCiphertext>,
    ) -> Result<Self, OfferCreationFailed> {
        // Random blinding factor for Pedersen commitment
        let rc_e: EmbeddedFr = rng.r#gen();
        let rc = Fr::try_from(rc_e)
            .expect("Fr should be within EmbeddedFr");

        // Compute coin commitment
        let coin_com = coin.commitment(&recipient);

        // Compute Pedersen value commitment
        let value_commitment = Pedersen::commit(
            &(coin.type_, segment.unwrap_or(0)),
            &coin.value.into(),
            &rc_e,
        );

        // Build proof preimage (witness data for ZK proof)
        let mut inputs = Vec::new();
        recipient.field_repr(&mut inputs);
        coin.field_repr(&mut inputs);
        inputs.push(rc);

        let proof_preimage = ProofPreimage {
            inputs,
            /* ... transcript ops ... */
            key_location: KeyLocation(
                Cow::Borrowed("midnight/zswap/output")
            ),
        };
```

```
        Ok(Output {
            coin_com,
            value_commitment,
            contract_address: match recipient {
                Recipient::Contract(addr) => Some(Sp::new(addr)),
                _ => None,
            },
            ciphertext: ciphertext.map(|x| Sp::new(x)),
            proof: Arc::new(proof_preimage),
        })
    }
}
```

Creating an Input (Spending a Coin)

```
// Source: zswap/src/construct.rs

impl<D: DB> Input<ProofPreimage, D> {
    pub(crate) fn new_from_secret_key<
        A: Debug + Storable<D>,
        R: Rng + CryptoRng + ?Sized,
    >(
        rng: &mut R,
        coin: &QualifiedCoinInfo,
        segment: Option<u16>,
        sk: SenderEvidence<'_>,
        tree: &MerkleTree<A, D>,
    ) -> Result<Self, OfferCreationFailed> {
        // Random blinding factor
        let rc_e: EmbeddedFr = rng.r#gen();
        let rc = Fr::try_from(rc_e)
            .expect("Fr should be larger than EmbeddedFr");

        // Compute nullifier from coin + secret key
        let nullifier = CoinInfo::from(coin).nullifier(&sk);

        // Compute Pedersen value commitment
        let value_commitment = Pedersen::commit(
            &(coin.type_, segment.unwrap_or(0)),
            &coin.value.into(),
            &rc_e,
        );

        // Get Merkle root
        let merkle_tree_root = tree.root()
            .ok_or(OfferCreationFailed::TreeNotRehashed)?;

        // Build proof preimage:
        // witnesses = [sk, merkle_path, coin_info, blinding_factor]
        let mut inputs = Vec::new();
        sk.field_repr(&mut inputs);                          // Secret key
        tree.path_for_leaf(coin.mt_index, ((), hash))        // Merkle path
            .map_err(OfferCreationFailed::InvalidIndex)?
            .field_repr(&mut inputs);
        CoinInfo::from(coin).field_repr(&mut inputs);        // Coin info
        inputs.push(rc);                                      // Blinding factor
```

```
        let proof_preimage = ProofPreimage {
            inputs,
            /* ... transcript ops ... */
            key_location: KeyLocation(
                Cow::Borrowed("midnight/zswap/spend")
            ),
        };

        Ok(Input {
            nullifier,
            value_commitment,
            contract_address: match sk {
                SenderEvidence::Contract(addr) =>
                    Some(Sp::new(addr)),
                _ => None,
            },
            merkle_tree_root,
            proof: Arc::new(proof_preimage),
        })
    }
}
```

The ZK Proof Flow

The `ProofPreimage` contains all the witness data needed to generate a zero-knowledge proof. The actual proof generation happens separately (via a proving key loaded from `spend.prover` or `output.prover`).

For spending (Input):

- **Public:** nullifier, value_commitment, merkle_root
- **Private (witness):** secret_key, merkle_path, coin_info, blinding_factor
- **Proves:** "I know a secret key and coin that produces this nullifier, and the coin's commitment exists at a leaf that hashes up to this merkle root"

For creating (Output):

- **Public:** coin_commitment, value_commitment
- **Private (witness):** recipient, coin_info, blinding_factor
- **Proves:** "I know a coin whose commitment matches, and its value commitment is correctly formed"

Chapter 14: The Offer Structure

An `Offer` packages inputs, outputs, and transients into a single atomic operation.

The Structures

```
// Source: zswap/src/structure.rs

pub struct Input<P: Storable<D>, D: DB> {
    pub nullifier: Nullifier,
    pub value_commitment: Pedersen,
```

```
    pub contract_address: Option<Sp<ContractAddress, D>>,
    pub merkle_tree_root: MerkleTreeDigest,
    pub proof: Arc<P>,
}

pub struct Output<P: Storable<D>, D: DB> {
    pub coin_com: Commitment,
    pub value_commitment: Pedersen,
    pub contract_address: Option<Sp<ContractAddress, D>>,
    pub ciphertext: Option<Sp<CoinCiphertext, D>>,
    pub proof: Arc<P>,
}

pub struct Transient<P: Storable<D>, D: DB> {
    pub nullifier: Nullifier,
    pub coin_com: Commitment,
    pub value_commitment_input: Pedersen,
    pub value_commitment_output: Pedersen,
    pub contract_address: Option<Sp<ContractAddress, D>>,
    pub ciphertext: Option<Sp<CoinCiphertext, D>>,
    pub proof_input: Arc<P>,
    pub proof_output: Arc<P>,
}

pub struct Delta {
    pub token_type: ShieldedTokenType,
    pub value: i128,
}

pub struct Offer<P: Storable<D>, D: DB> {
    pub inputs: Array<Input<P, D>, D>,
    pub outputs: Array<Output<P, D>, D>,
    pub transient: Array<Transient<P, D>, D>,
    pub deltas: Array<Delta, D>,
}
```

Transient Coins

A transient coin is created and spent in the **same transaction**. It has both a commitment (output side) and a nullifier (input side). Use case: intermediate coins in multi-hop contract interactions.

```
// Source: zswap/src/local.rs

pub fn spend_from_output<R: Rng + CryptoRng + ?Sized>(
    &self,
    rng: &mut R,
    secret_keys: &SecretKeys,
    coin: &QualifiedCoinInfo,
    segment: Option<u16>,
    output: Output<ProofPreimage, D>,
) -> Result<(State<D>, Transient<ProofPreimage, D>), OfferCreationFailed> {
    // Create a temporary Merkle tree with just this output
    let tree = MerkleTree::blank(ZSWAP_TREE_HEIGHT)
        .update_hash(0, output.coin_com.0, ())
        .rehash();
    // Spend from this temporary tree
    let (res, input) = self.spend_from_tree(
        rng, secret_keys, coin, segment, &tree
```

```
    )?;
    // Combine output + input into a Transient
    let io = Transient {
        nullifier: input.nullifier,
        coin_com: output.coin_com,
        value_commitment_input: input.value_commitment,
        value_commitment_output: output.value_commitment,
        contract_address: output.contract_address,
        ciphertext: output.ciphertext,
        proof_input: input.proof,
        proof_output: output.proof,
    };
    Ok((res, io))
}
```

Deltas: The Balance Sheet

Deltas track the net value flow per token type:

- **Positive delta** = more spent than created (value flowing out)
- **Negative delta** = more created than spent (value flowing in)

For a balanced transaction, all deltas should be zero (unless there's an intentional unshielding/shielding conversion).

Chapter 15: HD Wallet Derivation

The TypeScript wallet uses BIP-32 HD key derivation to generate the Zswap seed.

Path Structure

```
// Source: midnight-wallet/packages/hd/src/HDWallet.ts

const PURPOSE = 44;
const COIN_TYPE = 2400;

// Path: m/44'/2400'/{account}'/{role}/{index}

export const Roles = {
    NightExternal: 0,    // Receiving NIGHT tokens
    NightInternal: 1,    // Change addresses
    Dust: 2,             // Dust token operations
    Zswap: 3,            // Shielded key material ← THIS ONE
    Metadata: 4,         // Wallet metadata
} as const;
```

How the Wallet Derives Keys

```
// Source: midnight-wallet/packages/hd/src/HDWallet.ts

export class HDWallet {
    private readonly rootKey: HDKey;

    static fromSeed(seed: Uint8Array): HDWalletResult {
        try {
            const rootKey = HDKey.fromMasterSeed(seed);
            return { type: 'seedOk', hdWallet: new HDWallet(rootKey) };
        } catch (e: unknown) {
            return { type: 'seedError', error: e };
        }
    }

    public selectAccount(account: number): AccountKey {
        return new AccountKey(this.rootKey, account);
    }

    public clear(): void {
        this.rootKey.wipePrivateData();  // Security: wipe when done
    }
}

export class RoleKey {
    public deriveKeyAt(index: number): DerivationResult {
        const path =
            `m/${PURPOSE}'/${COIN_TYPE}'/${this.account}'/${this.role}/${index}`;
        const derivedKey = this.rootKey.derive(path);
        return derivedKey.privateKey
            ? { type: 'keyDerived', key: derivedKey.privateKey }
            : { type: 'keyOutOfBounds' };
    }
}
```

For shielded operations:

```
m/44'/2400'/0'/3/0  →  32-byte derived key
                            ↓
                       Used as Seed for Rust key derivation
                            ↓
                       coin_sk + enc_sk
```

The HD path `m/44'/2400'/0'/3/0` means: BIP-44 / Midnight coin type / account 0 / Zswap role / index 0.

Chapter 16: Sync Architecture

The TypeScript wallet SDK provides the sync layer that connects to the indexer and feeds events into the Rust state machine.

The WebSocket Subscription

```
// Source: midnight-wallet/packages/indexer-client (GraphQL)

// The GraphQL subscription:
// subscription ZswapEvents($id: Int!) {
//     zswapLedgerEvents(id: $id) {
//         id
//         raw       ← hex-encoded serialized Event
//         maxId
//     }
// }
```

SecretKeysResource: One-Time-Use Pattern

```
// Source: midnight-wallet/packages/shielded-wallet/src/v1/Sync.ts

export const SecretKeysResource = {
    create: (secretKeys: ledger.ZswapSecretKeys): SecretKeysResource => {
        let sk: ledger.ZswapSecretKeys | null = secretKeys;
        return (cb) => {
            if (sk === null) {
                throw new Error('Secret keys have been consumed');
            }
            const result = cb(sk);
            sk = null;  // Keys consumed — can't be used again
            return result;
        };
    },
};
```

This ensures secret keys are only accessible once per update batch, preventing accidental reuse.

Event Batching and Throttling

```
// Source: midnight-wallet/packages/shielded-wallet/src/v1/Sync.ts

return pipe(
    ZswapEvents.run({ id: Number(appliedIndex) }),
    Stream.provideLayer(WsSubscriptionClient.layer({ url: indexerWsUrl })),
    Stream.mapError((error) => new SyncWalletError(error)),
    Stream.mapEffect((subscription) =>
        pipe(
            subscription.zswapLedgerEvents,
            Schema.decodeUnknownEither(EventsSyncUpdateFromPayload),
            Either.mapLeft((err) => new SyncWalletError(err)),
            EitherOps.toEffect,
        ),
    ),
    // Batch: collect up to 10 events within 1ms windows
    Stream.groupedWithin(batchSize, Duration.millis(1)),
    Stream.map(Chunk.toArray),
```

```
    // Wrap each batch with secret keys
    Stream.map((data) => WalletSyncUpdate.create(data, secretKeys)),
    // Emit batches with 4ms spacing
    Stream.schedule(Schedule.spaced(Duration.millis(4))),
);
```

Two-stage throttling:

1. **Collection:** Gather up to 10 events within a 1ms window
2. **Emission:** Space out batch emissions by 4ms

TypeScript CoreWallet: State + CoinHashes

```
// Source: midnight-wallet/packages/shielded-wallet/src/v1/CoreWallet.ts

export type CoreWallet = Readonly<{
    state: ledger.ZswapLocalState;
    publicKeys: PublicKeys;
    protocolVersion: ProtocolVersion.ProtocolVersion;
    progress: SyncProgress;
    networkId: string;
    coinHashes: CoinHashesMap;
}>;
```

The `CoinHashesMap` is a precomputed cache:

```
// Source: CoreWallet.ts

export type CoinHashesMap = Readonly<
    Record<ledger.Nonce, {
        nullifier: ledger.Nullifier;
        commitment: ledger.CoinCommitment;
    }>
>;
```

This avoids recomputing commitments and nullifiers every time — they're expensive SHA-256 operations.

The applyUpdate Flow

```
// Source: Sync.ts — makeEventsSyncCapability

applyUpdate: (state: CoreWallet, wrappedUpdate: WalletSyncUpdate)
    : CoreWallet =>
{
    if (wrappedUpdate.updates.length === 0) {
        return state;
    }

    const lastUpdate = wrappedUpdate.updates.at(-1)!;
    const nextIndex = BigInt(lastUpdate.id);
    const highestRelevantWalletIndex = BigInt(lastUpdate.maxId);
```

```
    // Skip if we've already processed this
    if (nextIndex <= state.progress.appliedIndex) {
        return CoreWallet.updateProgress(state, {
            highestRelevantWalletIndex,
            isConnected: true,
        });
    }

    // Use secret keys exactly once to replay events
    return wrappedUpdate.secretKeys((keys) => {
        return CoreWallet.updateProgress(
            CoreWallet.replayEvents(
                state,
                keys,
                wrappedUpdate.updates.map((u) => u.event),
            ),
            {
                highestRelevantWalletIndex,
                appliedIndex: nextIndex,
                isConnected: true,
            },
        );
    });
}
```

Appendix A: Type Quick-Reference

Rust Type	Wraps	Size (bytes)	Field Elements	Used For
`Fr`	`outer::Scalar`	32	1	Primary field element
`EmbeddedFr`	`embedded::Scalar`	32	—	Jubjub scalar (Pedersen randomness, enc secret key)
`EmbeddedGroupAffine`	`embedded::Affine`	32	2	Jubjub point (enc public key, Pedersen commitment)
`HashOutput`	`[u8; 32]`	32	2	SHA-256 output
`Nonce`	`HashOutput`	32	2	Coin uniqueness
`ShieldedTokenType`	`HashOutput`	32	2	Token identifier

Rust Type	Wraps	Size (bytes)	Field Elements	Used For
`coin::SecretKey`	`HashOutput`	32	2	Coin ownership proof
`coin::PublicKey`	`HashOutput`	32	2	Coin recipient identifier
`Nullifier`	`HashOutput`	32	2	Spend proof
`Commitment`	`HashOutput`	32	2	Existence proof
`CoinInfo`	`{nonce, type_, value}`	80	5	Unqualified coin
`QualifiedCoinInfo`	`{nonce, type_, value, mt_index}`	88	6	Coin with tree position
`Pedersen`	`EmbeddedGroupAffine`	32	2	Homomorphic value commitment
`CoinCiphertext`	`{c: Point, ciph: [Fr; 6]}`	224	8	Encrypted coin
`encryption::SecretKey`	`EmbeddedFr`	32	—	Decryption key
`encryption::PublicKey`	`EmbeddedGroupAffine`	32	—	Encryption key

Appendix B: Domain Separators

Every cryptographic operation uses a unique domain separator to prevent cross-protocol attacks.

Domain Separator	Used In	Hash Function
`"midnight:csk"`	Coin secret key derivation	SHA-256
`"midnight:esk"`	Encryption secret key KDF	SHA-256 (multi-round)
`"midnight:zswap-pk[v1]"`	Coin public key derivation	SHA-256
`"midnight:zswap-cc[v1]"`	Coin commitment	SHA-256
`"midnight:zswap-cn[v1]"`	Coin nullifier	SHA-256

Domain Separator	Used In	Hash Function
`"midnight:field_hash"`	hash_to_field	Poseidon
`"midnight:schnorr_challenge"`	PureGeneratorPedersen	SHA-256
`"mdn:lh"`	Merkle tree leaf hash	—
`"midnight/zswap/spend"`	Input ZK proof key	—
`"midnight/zswap/output"`	Output ZK proof key	—
`"midnight/zswap/sign"`	Authorized claim ZK proof key	—

This document is a companion to `SHIELDED_BALANCE_DEEP_DIVE.md`. That document explains the concepts. This one shows the code. Together they provide complete onboarding material for implementing shielded operations in the Kuira Android Wallet.